Rldcoin: Interstellar Peer-to-Peer Payments

Runlai Deng

dengrunlai@gmail.com · www.rldcoin.com

19 October 2026

Abstract. A payment system spanning worlds cannot require every purchase to wait for a distant vote. We describe one native currency, runlai (RLD), whose spend authority is partitioned before disconnection and transferred through authenticated evidence. Each output has one authoritative home. Local Byzantine consensus resolves conflicting spends; independently verified execution checks monetary validity. Destinations preissue source-bound receiving capabilities with funded limits, leaving the payer to select actual inputs locally. Immutable exports can then travel forward without a payment-specific preparation round trip. Public work distributes a fixed twenty-one-million-RLD budget through disjoint slot entitlements. Placement generations are separate from Mint cursors, so ordinary mining cannot invalidate a receiving capability or an already detached range. Amounts coalesce by identical authority-exposure classes rather than individual mining events. Recursive execution receipts summarize historical validity while complete current spend data remains available. We specify these transitions, proof obligations, conservation and resource boundaries. The construction permits local circulation during inter-domain isolation and onward spending without a return receipt. It does not eliminate accepted ordering authorities, proof-system assumptions, unavailable data, destroyed quorums, finite funding or physical communication delay.

1. A payment system for separated worlds

A community should be able to pay a local worker while another world is unreachable. A ship should retain its prepared economy after departure. An authenticated invoice should let the payer choose inputs later, without sending a payment draft across space and waiting for permission to use those particular coins. A valid payment arriving through a succession of contacts should not need another interstellar round trip before its recipient can spend it. These requirements concern both communication and the location of conflict authority.

A globally preferred greatest-work history can eventually discard a competing branch [1]. It cannot also promise that every independently advancing branch of an indefinitely divided economy will remain final. We therefore assign each spendable output one home before use. Distinct homes can continue with distinct funds. Two partitions of the same home cannot both acquire its balance by declaring themselves independent.

The currency is runlai, ticker RLD, for principal, native fees and service escrows. Moving funds creates neither a wrapped currency nor a domain-issued substitute. The lifetime cap is 21,000,000 RLD, with zero spendable genesis supply and zero founder, investor, treasury or validator allocation. Denomination does not establish purchasing power.

The payment claim has two parts. Monetary execution can be independently checked: input ownership, issuance work, amounts, fees and authority transfers follow public rules. Exclusive ordering still needs an accepted conflict authority: a proof that a transaction is valid does not prove that another valid transaction spending the same input was never certified elsewhere. Separating these statements avoids treating either a signature or a validity proof as a solution to every fault.

This is peer-to-peer payment in the sense that owners authorize exact payments, any willing party may carry evidence, and there is no compulsory central payment intermediary or continuously reachable Earth directory. Local settlement uses explicitly admitted validators. Automatic trust in arbitrary new committees and universal permissionless consensus are not supplied by the term peer-to-peer.

2. Model, identities and assumptions

A domain is an authenticated replicated state machine, not a geographical monopoly. Several domains may coexist on a planet, and a ship can form its own domain. A stable descriptor binds the common currency identity C_RLD, initial validators, consensus and encoding rules, cryptographic suites, execution-proof profiles, resource bounds, fee schedules and authorized succession. A self-asserted location or relay address is not that descriptor.

Identifiers are constructed before the objects that sign them. A stable domain ID hashes an initial identity statement containing an identity public key and nonce, not its later configuration, key trees or its own digest. The monetary descriptor names D_0 by that identity and fixes the initial logical slot intervals; C_RLD excludes subsequent certificates and state roots computed from C_RLD. Initial configuration attestations and key-tree roots bind those already fixed identifiers. A recipient's accepted anchor authenticates the complete initial descriptor; a second configuration claiming the same identity is not silently adopted. This ordering avoids circular genesis, key-tree and certificate commitments.

The baseline local consensus is PBFT with n = 3f + 1 distinct admitted validators, the prepare, commit, view-change and new-view rules of [2], and at least 2f + 1 distinct commit messages for a finalized sequence position. Messages additionally bind C_RLD, the stable domain, configuration, sequence, view, exact operation-batch digest, predecessor state and checkpoint header H defined in Section 4.1. Ordinary owner operations are deterministic PBFT commands. A signed state root without the protocol's commit conditions is not finality. A different consensus suite requires a separately identified profile and authenticated adoption, not a local reinterpretation of a certificate.

Every relevant configuration assumes at most f Byzantine validators, correct deterministic validation, authentic keys, collision-resistant hashes and durable non-rollback consensus state. Safety does not require a timely stellar response. Liveness requires a responsive local quorum, eventual communication conditions sufficient for PBFT, fair bounded admission and funded resources. These are obligations of the accepted domain, not consequences of a radio connection.

An owner may control several keys, carriers may duplicate or lose messages, and an adversary may reorder contacts, lie about routes, submit conflicting spends or withhold evidence indefinitely. Owners need not be online simultaneously. A minority partition waits. A single surviving validator has no authority to replace a failed quorum; a deliberately single-validator domain has no Byzantine replication tolerance.

Receiving policies select acceptable configuration histories and maximum exposures before accepting funds. An available descriptor with identical rules is not automatically a trusted ordering authority. Threshold compromise, including applicable historical-key compromise, remains outside the safety claim. A finite cryptographic horizon, sound execution proofs for the specified monetary program, available current state and an eventual physical path are prerequisites of the corresponding verification and delivery claims.

3. Money with one authoritative home

One RLD equals 10^24 atomic runlai units. Amounts are nonnegative bounded integers; no floating-point rounding participates in authorization. C_RLD commits to one authenticated monetary genesis, the denomination, the complete issuance budget, work rules, retirement rules and succession restrictions. Matching a name or ticker is insufficient. A rule change cannot create a second supply under another identifier and call both balances the same RLD.

A spendable output binds its identifier, currency, owner policy, home domain and a normalized map from dependency class to integer amount. A class is the exact canonical set of authority/configuration/key-policy exposures on which that amount depends. Its content hash identifies the set; the hash is not a substitute for the set's available contents. Equal classes coalesce by adding amounts, regardless of their distinct Mint origins. Unequal classes remain separate. Components are accounting parts of RLD, not new currencies, and their sum is the output amount. Only the current home can consume the output. Every actual input owner signs the complete allocation to recipients, change, explicit service fees and retirement. Section 9 defines this lossless grouping.

The local state contains unspent outputs U, exports E, permanent destination decisions J, receiving reservations R, source capability-use decisions S, issuance intervals A, configuration history and service escrows. All maps are committed by

the executed state. No public observation of a remote balance is signing authority. Every operation has a typed action tag, exact predecessor and unique identifier. Payment, capability, issuance and service-object namespaces are distinct.

3.1. A fixed, divisible issuance budget

The cap is C = 21,000,000 RLD = 21 x 10^30 atomic units. There are N = 4096 budget roots, each containing L = 1,000,000 slot identities. Slot (s,j), with 0 <= s < N and 0 <= j < L, can pay rho = 0.005126953125 RLD = 5126953125000000000000 atomic units once. N times L times rho equals C exactly. All successful Mint rewards go to the bound work author. There is no presale, reserve withdrawal, validator subsidy, halving, tail emission or replacement issuance for lost or retired money.

An active allocation tile is (s,g,lo,hi,k,home,placement_anchor,mint_anchor), with lo <= k <= hi. The immutable generation g identifies this placement of [lo,hi); k is the next unconsumed slot. Its remaining budget is (hi-k) rho. Ordinary Mint advances k and mint_anchor but preserves g and placement_anchor. Splitting, exporting, installation and recovery instead create new allocation generations. These two kinds of change must not share one mutable identity.

A claim may detach any still-unconsumed subrange [a,b) with k <= a < b <= hi and b-a <= 256. The source atomically closes g, exports precisely [a,b), and retains nonempty disjoint residual tiles [k,a) and [b,hi), each with its own new generation and initial cursor. The already consumed prefix [lo,k) remains consumed. No descendant overlaps another or restores a consumed slot. This baseline has no interval merge operation.

Thus a root's disjoint ranges may reach more than 4096 prepared issuance homes while the total payable slot count remains 4,096,000,000. A domain obtains no budget merely by existing or claiming a geographical name. A moved range is an entitlement to perform later Mint work, never an RLD output or a second asset supply.

3.2. Public placement rather than discretionary awards

Initially the nonspendable root tiles (s,g_s,0,L,0,D_0,...) reside in the genesis-designated bootstrap domain D_0. Choosing that initial quorum remains an explicit trust and availability decision. It has no administrative command to award issuance beneficiaries, pay out the budget or replace public work with a private eligibility list. Initial and later placement use the same ClaimRange rule.

A receiving domain first supplies a typed ReceiveCapability under Section 5. For kind RLD_ISSUANCE_INTERVAL it commits to the source, destination, acceptable roots and placement dependencies, maximum slot count, proof/resource bounds, service funding and claimant-use policy. It does not bind a source cursor, current head, exact parent tile or range not yet selected. The claimant can therefore obtain this capacity before solving placement work or observing a future source state.

Placement work binds the stable generation g and placement_anchor, the selected [a,b), destination, capability digest and claimant policy. It excludes the mutable Mint cursor, mint_anchor and source head. At source finalization, g must still be active and the selected slots unconsumed. Earlier Mint of slots below a changes neither placement work nor receiving capacity. If Mint consumes a selected slot before the claim finalizes, or another claim closes g, that local proposal loses and must be recomputed. There is no promise of conflict-free local work. The unused generic capability remains usable for another conforming proposal without a stellar renewal.

The claimant supplies the capability-use authorization as well as signed placement work. As with payment export, the source prepares the candidate receipt and complete current material before the claim can finalize. The first valid finalized claim atomically performs the exact split in Section 3.1, sets S[p] = Exported(x), and records an immutable range ticket Q_x = (s,[a,b),destination,lineage,transition). Source Mint can no longer consume those slots. Minting in any residual tile cannot change Q_x or its receiving terms. The source never waits for a destination to approve that now-changing exact range: it already has a generic obligation covering it.

The destination verifies the certified range export and installs exactly [a,b) with cursor a, or permanently rejects it. Installation consumes R[p] and its permanent move decision together. Only certified rejection permits one source recovery of the same detached slots, with a new generation and unchanged lineage. Recovery cannot merge them into an

overlapping parent. A later onward move does not erase the prior decision. No RLD is issued by claim, carriage, installation or recovery; any service payment uses separately funded existing RLD.

A child generation hashes its consumed parent generation, exact child bounds, role and transition-body digest. The body binds the old placement anchor, selected range and capability but excludes resulting IDs, state roots and new anchors. New placement anchors derive from that body and role; Mint anchors then evolve separately. This construction avoids circular commitments and prevents moving a range from resetting its slot identities.

Public work removes discretionary beneficiary selection, not hardware advantage, censorship or intentional stranding. Work scales with detached slots; one cheap solution cannot seize an arbitrarily large budget. A claim requires no prior RLD balance and earns no RLD reward. The maximum of 256 bounds one request, not a per-person share or a proven optimum. Bootstrap availability, fair admission and funded receiving capacity remain dependencies. Lost in-flight entitlements reduce attainable issuance; timeout never recreates them.

3.3. Public work and Mint

Mint and ClaimRange use Argon2id version 0x13 [3], memory 65536 KiB, three passes, one lane, output 32 bytes, empty secret and associated data. Enc is unambiguous typed encoding. Mint binds the exact current tile, k and mint_anchor; ClaimRange binds the stable generation, placement_anchor and chosen range instead. Both bind currency, source home, author policy and a 32-byte nonce. ClaimRange also binds destination and capability digest. Define:

```
z_mint = enc(C_RLD, "Mint", s,g,lo,hi,k,home,mint_anchor,
             payout_policy, nonce)
z_claim = enc(C_RLD, "ClaimRange", s,g,lo,hi,home,
              placement_anchor,a,b,destination,capability_digest,
              claimant_policy,nonce)
z = z_mint or z_claim for its typed action
P = SHA256(enc("RLD-WORK-PASSWORD", z))
S = first16(SHA256(enc("RLD-WORK-SALT", z)))
y = Argon2id(P,S; m=65536,t=3,p=1,v=0x13,out=32)
weight = 1 for Mint; b-a for ClaimRange
T_base = 2^240 - 1
valid_work = big_endian_integer(y) <= floor(T_base / weight)
```

The work author signs the complete candidate. Copying a solution cannot redirect its payout or placement. Distinct tags prevent a placement solution being used as Mint. Under a uniform-result model, Mint succeeds with probability 2^-16 per evaluation, or 65,536 evaluations on average. A claim covering n slots has probability (floor(T_base/n)+1)/2^256 and expected work approximately n times that of Mint. Placement work cannot replace later slot work. These are work counts, not time guarantees, hardware benchmarks, ASIC resistance or ordinary-CPU profitability. There is no stake, compulsory pool or initial RLD payment for valid work submission; verification still needs bounded sponsored resources and fair admission.

```
Mint(candidate):
  require exact active generation g, cursor k and mint_anchor
  require k < hi, author signature and valid Mint work
  atomically create rho for author; record unique (C_RLD,s,k)
  set k = k+1; advance mint_anchor using the transition body
  preserve g, lo, hi and placement_anchor; retire tile if exhausted
```

The first Mint anchor of a new tile derives from its creation statement. Later Mint anchors hash the predecessor and exact Mint body, excluding resulting state roots and new anchors. An old Mint candidate cannot follow another Mint or move to another generation/home. Ordinary payments and validator succession do not reset issuance identity. A proposal, solved hash or preliminary vote is not issued money.

There is no automatic difficulty retarget or calendar completion date. Hardware progress can accelerate exhaustion. Local governance cannot weaken work, alter rho or replenish slots. If confidence in the primitive fails, affected issuance stops; a replacement requires authenticated succession preserving every consumed identity and preventing old/new double exercise. Scarcity alone establishes neither equal distribution nor purchasing power.

4. Local settlement and its meaning

Owners sign exact currency and domain identities, inputs, class allocations, recipients, amounts, fees, service obligations, nonce and protocol/suite versions. Validators execute these deterministic operations before PBFT voting. A local recipient verifies the commit proof, output inclusion and monetary execution, either by maintaining fully validated state or by checking the execution receipt described below. Submission acknowledgment and wallet reservation are not settlement.

A settled output is usable in the next valid local transition. No global block, mining maturity or stellar receipt is required. A domain can settle existing RLD without mining. Computation, local communication, resource admission and durable storage determine latency. A device that verifies only a root without its specified proof and current output data has not independently checked the money.

Full causal replay defines reference semantics. Starting from C_RLD and authenticated descriptors, it executes owner authorization, work, interval partition, class conservation, capability use, import/rejection exclusion, fees and succession. Imports verify prior certified source execution. Missing dependencies, cycles, foreign currencies and conflicting predecessors refuse. For carriage to cold receivers, recursive execution receipts replace retransmission and replay of the entire prefix; they do not replace these semantics with trust in a snapshot.

4.1. Execution receipts and certified checkpoints

A checkpoint header H commits to C_RLD, proof-profile ID, domain, sequence, predecessor certificate ID, predecessor and resulting state roots, operation-batch digest, active signing configuration, successor-configuration root if any, and an outbound-material manifest root. That manifest binds the ticket, normalized class/tag tables and current data, excluding the enclosing header, receipt and QC to avoid a circular commitment. Counters, canonical encoding and declared resource limits are included in the state/profile binding. The exported output or decision is proven included in these exact commitments. State roots cover U, E, J, R, S, issuance, configurations, exposures and funded service liabilities, not merely balances.

An execution receipt proves a candidate transition to H. It does not prove its own finality. A separate PBFT commit certificate QC signs H; neither H nor its operations contain the receipt's bytes or its own QC. The receiver checks both a successful execution receipt with journal exactly H and the matching QC under the proven active configuration. The predecessor certified checkpoint, not a prover-chosen state root, anchors continuation.

```
Exec(H; witness):
  base: authenticate the common zero-issuance genesis;
        or derive a new domain's zero state and admitted identity
  step: verify predecessor execution receipt AND its exact QC
        reconstruct authenticated current state/access witnesses
        execute every intervening certified native batch in order
        verify each imported receipt, QC and available current data
        execute candidate batch under full reference monetary rules
  require computed header = H and all recursive assumptions resolved
  publish H as the receipt's exact public journal
Certified(H) = VerifyExecReceipt(H) AND VerifyMatchingQC(H)
```

A new domain has no initial balances or issuance; it can acquire them only through valid source exports. Identity acceptance remains a receiving policy, not proof-created trust. Configuration transitions verify the old authority and new acknowledgment; a certificate cannot choose its own signing keys by including them beside a signature. Imported certificates and predecessor receipts form a finite causal proof graph with strictly earlier dependencies; cyclic or self-assumed claims refuse.

A node may execute ordinary local batches directly between proof checkpoints. To build the next receipt it folds the entire contiguous certified suffix since its last receipt, including every QC, before executing the new candidate. Skipping an intermediate debit or handoff is invalid. For an outbound export, the candidate receipt and all required current material must be ready before the source commits the debit. The source may batch exports or hold a local admission barrier while proving. If the native head changes first, it retries the local candidate under the same unused capability. This can be

computationally expensive and does not promise a seconds-scale export. It avoids locking money first and discovering afterwards that no conforming proof can be built.

A later receipt verifies the preceding receipt and QC inside its proof relation. An outer verifier checks only the new receipt and matching current QC, plus current inclusion/material and acceptance policy. Full replay remains an audit and implementation-equivalence method; receiving an unverified snapshot or silently substituting an unsupported proof backend is not a fallback.

4.2. A pinned recursive proof profile

A reference realization is the RISC Zero zkVM 3.0.6 SuccinctReceipt profile, which uses recursive STARK receipts for long computations [17,18]. It is selected as an explicit construction boundary, not evidence that an RLD guest program already exists or that its proving cost is practical. The monetary descriptor must fix the exact verifier and trusted control root, hash function and parameters, guest program image IDs, successful-exit requirement, journal codec and limits before monetary use. Package version alone is not that binding. No unspecified future proof algorithm is implicitly accepted.

The monetary guest implements the reference transition relation and recursively verifies the predecessor and imported certified executions. Composition assumptions must be discharged by real subordinate receipts, producing a final unconditional receipt. A conditional receipt that merely claims another execution, a development/fake receipt, an unknown program or control root, a pruned unbound journal, an unsuccessful exit or unresolved assumption refuses. Seal-integrity verification alone does not check the intended program, complete claim, journal or acceptance policy. The public verifier must check all of them against independently authenticated profile parameters.

Proof soundness, correct guest implementation and correct cryptographic/encoding gadgets are additional premises. A proof establishes that its specified program executed; it does not prove that the program correctly captures this paper. Argon2 work, PBFT signatures and imported proofs must actually be checked inside the authenticated relation rather than delegated to an unchecked host assertion. Recursion can bound carried historical verification, but cannot make those computations free. Proving memory, cycles, receipt size and local queueing require their own measurements; no numerical performance claim follows from choosing this backend.

4.3. Validity, current data and ordering

A compact receipt authenticates historical validity, not data availability. A payment capsule must carry its exact ticket and capability, certified header and QC, inclusion paths, the complete normalized current component table and its deduplicated exposure sets, and current authority/key descriptors required by the agreed policy. The destination durably installs this material before crediting onward-spendable outputs. Its operators retain the state or access witnesses and receipts necessary to execute and prove the next transition without asking an unreachable source. Receipt-only arrival remains pending.

Proofs reject invalid ownership, inflation and split arithmetic under their soundness assumptions, even if all validators sign. They do not distinguish two individually valid conflicting certified histories during silence. Exclusive ordering still needs the fault-bounded authority and durable PBFT locks. A compromised quorum can certify two valid spends of the same old input; both programs may execute correctly on their separate branches. Known incompatible evidence triggers an incident; an unseen fork cannot be inferred from a proof.

Finality therefore means extension of the accepted domain's committed prefix under these premises. Reconnection alone cannot erase it by selecting a distant mining branch. Threshold compromise, rollback, invalid succession, lost data, unsound proofs and broken cryptography remain failures. A traveler must relocate funds before losing the old home's availability; a private key does not relocate authority.

5. Export: surrender before sending

A destination preissues a finite family of ReceiveCapability coupons, normally in an authenticated invoice or contact announcement. Each coupon binds C_RLD, exactly one source domain, the stable destination, recipient policy, object kind, maximum principal or permitted issuance roots/slot count, fee formulas, recovery constraints, an immutable acceptance-policy digest, supported proof profile and current-material/resource bounds. A use policy names a coupon-use public key or explicitly permits public use. This key authorizes the use of capacity; actual monetary input owners are chosen later and must separately sign the complete payment.

The coupon deliberately excludes actual inputs, change, exact principal below the cap, source height, Mint cursor, mutable anchors and an exact future issuance object. It authorizes one conforming source choice, not one previously drafted transaction. A payer holding an accepted coupon can assemble and prove a payment entirely at its source and then send it forward. Holding N coupons permits N distinct uses without N stellar preparation conversations. Coupons contain no RLD credit and do not prefund the recipient's principal.

A family identity hashes currency, source, destination, issuer nonce and identity statement, excluding its later leaf root. Leaves indexed 0 through N-1 commit to their coupon terms; p = (family_ID,index). A finalized family certificate binds that exact leaf root and funded N-use reservation. The destination's one-family record excludes another root under the same identity. A coupon carries its membership path and family certificate. Merkle compression reduces certificate repetition, not the N logical capacity liabilities. An individual coupon is a family of size one.

Binding exactly one source is essential. A wildcard usable at two independent sources could authorize two exports against one receiving slot. A multi-source service must issue disjoint coupon sets for the named sources and reserve their total capacity. A changed source needs its own coupon; route changes do not change monetary source or destination. Public-use coupons explicitly expose their finite slots to competing users; private invoices should bind an access key to prevent another payer occupying them first.

R[p] = Reserved is a funded resource obligation, not money. Before certifying a family, the destination accounts for bounded proof verification, complete current-material storage, terminal decisions and service funding for every leaf. Sponsors may fund recipients with no balance. A family's reservation cannot be spent again merely because its users remain silent. New capability issuance stops when capacity is exhausted; old obligations persist.

Coupons do not expire because a message is late. Closing an unused coupon requires its use-policy authorization and a certified source command CloseCapability(p), which checks S[p] absent and permanently sets S[p] = Closed. The destination verifies that execution before releasing R[p]. Otherwise a supposedly unused coupon might already back an in-flight export. A failed source can strand unused capacity; finite funded issuance limits that exposure but cannot erase it. Ordinary later fee or policy changes cannot revoke a conforming outstanding coupon.

```
x = H(enc("RLD-EXPORT", C_RLD,source,destination,T))
a = principal + destination_fee
Export(T,capability):
  require authentic family, leaf p, use authorization and S[p] absent
  require T satisfies source, recipient, kind, fees and all limits
  require all actual input owners sign T and class allocations
  require inputs cover a + source_fee + explicit change
  prepare valid candidate receipt and complete current material
  atomically finalize inputs, change, fees, S[p]=Exported(x),E[x]
```

T binds the coupon digest, chosen actual inputs, exact amount, fees, normalized allocations, recovery policy and nonce. The source's current monetary state chooses those inputs once. An unfinalized proposal, proof retry or changed local head consumes neither coupon nor money. The matching QC finalizes the debit; evidence release follows durable finalization. Lost responses reconcile against that exact source decision, never an assumed refund. Carriers cannot change recipient, fees or destination.

A coupon-compatible invoice can arrive before the payer owns its eventual inputs. Subsequent input changes or ordinary Mint do not require its reissuance. A wholly unknown destination, unsupported source, depleted family or changed

nonconforming requirements still needs setup communication or fresh capacity. The claim is one forward payment after a source-bound coupon is available, not communication-free discovery or faster-than-light information.

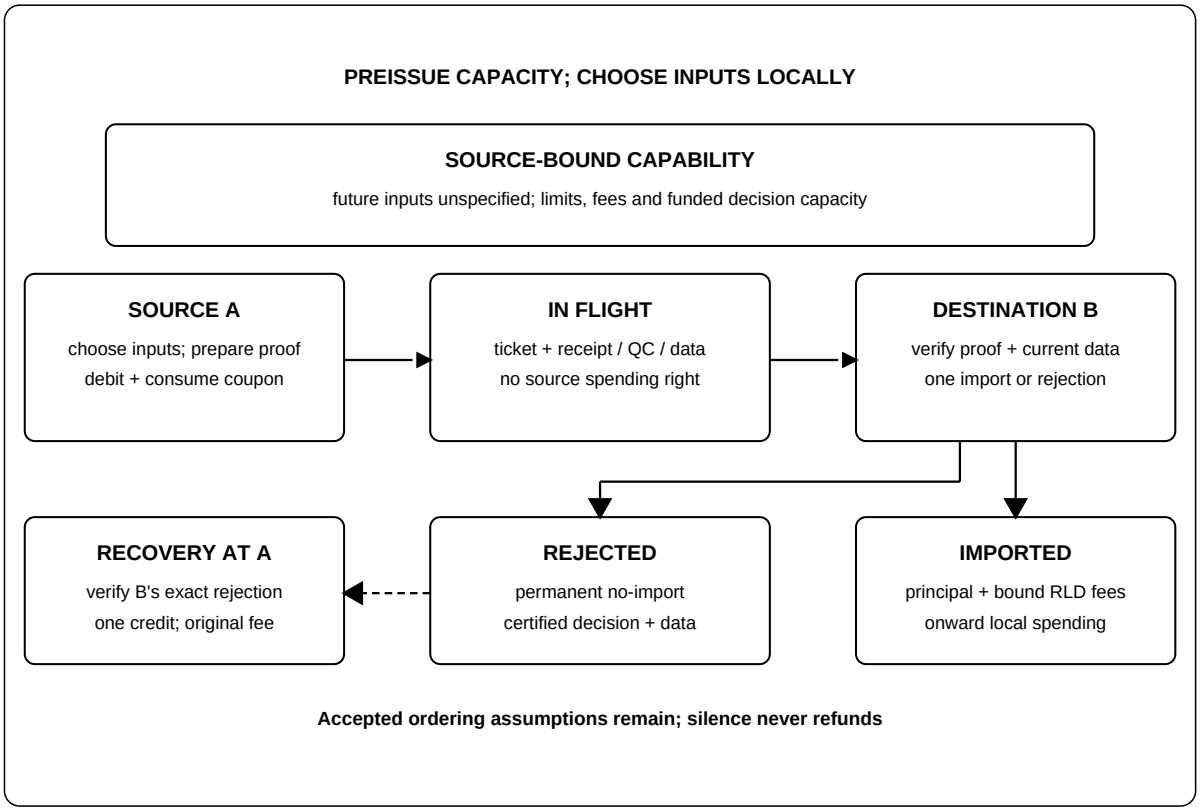


Figure 1. A source-bound receiving capability is available before inputs are chosen. A proof-ready export travels forward to one import or permanent rejection; only certified rejection allows recovery.

6. Import: a single destination decision

Any carrier may deliver T, owner/use signatures, its capability and membership path, source execution receipt, matching commit certificate, inclusion proofs and complete current material. The destination checks the pinned proof profile and journal, accepted source/configuration continuity, final source debit, S[p] = Exported(x), exact class allocation, every class against the acceptance predicate and absence of incompatible locally known evidence. A Merkle path proves inclusion, not valid money by itself.

If the reserved leaf remains undecided and this package meets the original terms, ordinary later policy or fee changes cannot revoke it. Correct validators process the outstanding obligation even after a newer configuration activates. Newly authenticated compromise, broken proof/cryptographic confidence or an out-of-bound/incomplete package is an explicit incident or pending obligation, not an invented refund certificate. The system cannot promise safe import of known conflicting money.

```
Import(T,capsule):
  require exact currency, capability p, destination and recipient
  require Certified(source_header), inclusion of Exported(T,a)
  require included S[p]=Exported(x), matching T and all class terms
  require R[p]=Reserved and J[x] absent; no known incident conflict
  durably retain complete bounded current material, not only roots
  atomically set R[p]=Imported(x),J[x]=Imported(H(T))
  create principal and authorized destination-fee outputs
```

Outputs retain the exact class allocation and add the destination's relevant ordering exposure to each contributing amount. Durable credit and durable consumption of p and x finalize together. Duplicate delivery returns the same result, never

another balance. Changed bytes under an existing identity, foreign destinations, unsupported proof profiles and mixed fragments refuse. Stable namespaces survive authenticated succession and compaction.

After import the recipient can spend locally or export onward without querying the source or receiving its acknowledgment. Its next proof uses retained current data and certified receipts, not retransmitted full origins. Optional return receipts improve the payer's knowledge and grant no source credit. Carrying A-to-B bytes through C adds no monetary authority; importing into C and exporting again is a distinct transfer requiring a coupon naming C as source.

7. Rejection, cancellation and stranded value

The destination serializes import and permanent rejection against both R[p] and J[x]. An authenticated rejection request binds the original export and recovery-owner authorization, or the recipient's authorization. If undecided, the destination may finalize Rejected. Once imported, it cannot validly reject; once rejected, it can never import. An invalid packet, unavailable server, expired transport envelope or full new queue is not a rejection certificate.

Recovery at the source verifies a destination execution receipt, matching QC, inclusion and current material proving that exact rejection, including the capability, authority succession and permanent terminal decision. It requires E[x] = Exported, changes it once to Refunded, and creates a-r for the recovery owner plus the originally authorized recovery fee r, with 0 <= r <= a. Component allocation is preserved; rejection adds its relevant ordering dependency. Separately funded rejection-service fees cannot silently consume another owner's principal.

A source cannot distinguish a never-delivered ticket from an already imported ticket whose receipt was lost. A rule refunding both cases would duplicate value in the latter. No elapsed-time refund exists. An import/rejection race is ordered by the destination, so a rejection request is not guaranteed recall. Rerouting requires permanent rejection, source recovery and a new owner-authorized export. The carrier cannot edit the destination.

A receiving capability removes avoidable policy and capacity surprises under continued correct operation; it cannot make a destroyed domain respond. If its quorum, archives or cryptographic assumptions permanently fail, money can remain unavailable. There is no automatic emergency replacement of the old authority. Independently operated replicas, prepared succession and off-domain copies reduce failure probability without changing the threshold assumption.

Optional insurance is separately funded in already issued RLD. A contract identifies the insurer, maximum payout, adjudicating authority, evidence and premium. On payout, it can transfer the original ticket's recovery/claim rights to the insurer through the authorized policy, avoiding a promise of both reimbursement and free later recovery to the same owner. Insurance does not mint a replacement coin or reverse the ticket; the insurer bears its finite exposure. It is additional credit and institutional risk, never a mandatory intermediary or guaranteed protocol rescue.

8. An interstellar payment and local autonomy

Bob's domain B supplies Alice an authenticated invoice with a coupon naming A as source. Alice need not already hold or reveal the future payment inputs. She may receive RLD later, choose compatible inputs locally, authorize the exact payment and finalize a proof-ready export. Carrier copies can then travel over radio, optical links, moving storage or successive one-way contacts. Alice can go offline after source finalization.

B verifies the recursive execution receipt, matching QC and all current material under the precommitted policy, then imports once. Bob obtains ordinary local RLD and can pay Carol without querying A. If B already holds a suitable coupon naming B as source at another destination, its onward export also needs no per-payment return preparation. A receipt to Alice can arrive years later without suspending Bob's economy.

```
In advance: B -> A: source-bound coupons; inputs are unspecified
At A: choose inputs; prepare proof; finalize export x using p once
Forward contacts: x, capability, receipt + QC + current material
At B: Reserved(p) -> Imported(x) OR permanently Rejected(x)
After import: Bob -> Carol by ordinary local settlement
After rejection: certified rejection returns -> one source recovery
```

A newly discovered pair may still exchange setup information, and a source-bound inventory can run out. The protocol eliminates payment-specific draft approval, not the initial need to know and accept a destination. Ships prepare domains, locally homed balances, coupon inventories, current data, proof capacity, archive/succession material and optional issuance ranges before isolation. Copying another ledger supplies no independent funds; a minority partition cannot declare its home balance locally spendable.

Where distance d must be crossed, new information cannot arrive before d/c. Contact waiting, serialization, proving, verification and destination consensus add delay. Prepositioned RLD makes local purchases possible; it does not accelerate new distant information. Separately funded liquidity can offer credit, but cannot call an unarrived export settled.

9. Provenance and the spread of trust

The monetary execution tracks each unique issuance ID (C_RLD,s,j), its valid work and placement history. That history must be verified when created and remains available for audit. It need not remain a separate live output component forever. A monetary risk class instead identifies the exact canonical set of immutable authority-exposure tags relevant to a particular amount. Tags bind domain, configuration/epoch and key/fault policy; a new genuinely different authority is a new tag. Per-height key offsets authenticate statements under that policy but are not new risk classes merely because another Mint occurred. A proof cannot relabel old exposure as a new trusted source.

For each class P, an operation partitions the incoming integer amount among outputs, change, fees, export and retirement. The operation adds the executing authority's applicable tag to each allocated part. At import it also adds destination exposure; rejection recovery adds the relevant rejection authority. After this deterministic transformation, amounts with exactly equal sets coalesce. Different sets stay separate: taking their union and attaching it to all money would invent exposure for otherwise unrelated portions. The complete normalized table is sorted uniquely and excludes zero entries and duplicate keys.

```
For every input class P:
  sum(authorized allocations from P) = total input amount in P
For each allocated part (P,v) under executing exposure e:
  output_class = canonical_set(P union {e})
  accumulate v only into that exact output_class
Mint: verify unique work slot; create rho under its full exposure set
```

Thus 100 RLD originating in 19,505 Mint events can be one amount entry if all those rewards have exactly the same exposure set. Grouping does not merge their issuance IDs or let a slot be minted twice: the execution proof checks slot uniqueness in the committed issuance state. If configuration or placement histories differ, there may instead be many classes. The optimization is by equal risk, never by an assumption that all RLD has equal risk. Two distinct sets cannot be combined just because a receiver currently accepts both.

An owner with 100 RLD in class P and 1 RLD in class Q can pay from P and leave Q in change. Proofs still check all inputs actually consumed. Accepted portions and unrelated change retain their respective contributors; display in one wallet changes nothing. The receiving policy evaluates every carried class and available tag set, not one favorable membership path. A hash of a class proves neither that all classes were shown nor that the receiver has the data to classify them.

Class count grows with genuinely different exposure sets, not necessarily with amount or individual Mint count. It is not bounded by a universal constant: long sequences of different authorities can still accumulate many tags. Coupons cap class count, tag count, current bytes and verification work before debit. An oversized amount may need partition into several accepted payments with sufficient coupons, a different acceptance agreement, or refusal. Splitting cannot delete real dependencies or guarantee acceptance.

RLD remains the only denomination and fee currency; differences in acceptance can still affect value. Wallets select invoice-compatible funds and distinguish accepted, other-policy, exported, insured and unresolved amounts. Shared policy versions can name proof profiles, accepted roots, exposure limits, fault-domain diversity and data availability. No self-created committee becomes trusted merely by proving valid execution.

An authenticated incident contains conflicting certificates or invalid execution and exact affected tags. Carriers propagate it causally. Policies may suspend new acceptance and quarantine dependent classes without erasing unrelated history or minting compensation. Unseen threshold equivocation remains undetectable from silence; a proof, waiting period or collateral cannot establish its absence. Finite exposure limits and funded insurance constrain named liabilities, not instantaneous economy-wide contagion.

10. Transport and durable evidence

The carriage model is store-carry-forward over actual contacts, compatible with the delay/disruption model of Bundle Protocol version 7 [4]. A transport envelope binds its version, currency/application domain, whole content hash, length, origin and destination. Fragments bind positions and exact whole-object commitment. A receiver reconstructs and authenticates the complete object before monetary processing. Transport expiry can remove an attempt, never a monetary obligation.

An admitted custodian durably stores complete agreed bytes before acknowledging custody. A digest without the bytes is not evidence retention. Capacity refusal precedes a new obligation. Custody contracts bind retention scope, replication, failure reporting, service fees and renewal responsibility. Independent copies reduce single-carrier loss but not correlated failure or fraud. A signed custody receipt proves an assertion of custody, not successful future delivery or destination settlement.

End-to-end authentication persists across route changes; link authentication secures the present contact [5]. Two-way contacts may negotiate missing dependencies. A one-way carriage package contains the receipt, QC and all current material needed by its declared verifier and next monetary execution, or relies on an explicitly arranged later material delivery. The latter remains pending until complete. Candidate advertisements cannot change a ticket's destination or supply a trusted domain root.

A request identifier, content commitment and durable receiver record make retries idempotent. Receipt loss resumes carriage without another debit. Ledger acceptance, byte retention, relay compensation and optional receipt return are separate events. No forwarding schedule, satellite range or unavailable physical route is invented by monetary consensus.

11. Fees, liquidity and incentives

Every native payment has an explicit positive RLD service fee. Its signed allocation specifies consensus operation, monetary verification and a positive archival-reserve contribution; a sponsor may pay on an owner's behalf through exact funded inputs. A work-author share, when required by a declared service profile, names a verifiable prior work event and payout policy and is a transfer of existing RLD. No label such as miner fee authorizes an unbound future deduction. Mint itself requires no RLD admission fee, so the first successful miner can obtain a balance without a circular funding prerequisite.

A destination capability family is issued only against a funded service agreement or sponsor budget covering its stated bounds. A zero-balance recipient may have a sponsor; that is a real payment by another party, not free perpetual capacity. Source export pays its exact source services; import credits the recipient and the bound destination services; recovery pays only its original formula. All sums and component assignments are owner-authorized. New fees cannot retroactively modify an exported ticket.

An archival reserve is existing RLD locked under an explicit threshold-controlled service policy. Records bind their remaining obligations, funded capacity and custodian set. Scheduled releases pay documented service under that policy; changing custodians requires authenticated handoff of complete data and liabilities. New capability families stop when funded capacity is exhausted, while existing terminal decisions and liabilities remain retained. A reserve cannot be withdrawn as surplus while its covered obligations persist. Without an independently specified proof-of-storage construction, releases still trust the agreed service authority; a receipt alone proves neither storage nor indefinite solvency.

Relay contracts are finite purchased services, also in RLD. Prepayment exposes a customer to nonperformance; delayed payment exposes a carrier to credit and return-path risk. Funded escrow, reputation, collateral and disclosed adjudication

may distribute these risks. An arbitrary one-way message is not an automatically fair exchange of delivery and payment. No new subsidy is created by uptime, receipts, self-payments or newly created identities.

Mining rewards distribute the fixed budget and do not prove that validators, carriers or archives can operate profitably. After issuance ends, they rely on service fees, voluntary support or other disclosed funding. Outstanding capability liabilities must not assume future inflation. No finite reserve buys mathematically infinite storage; continued service needs funded renewal and succession. Failure of that economic premise can stop progress without granting a refund or rewriting the cap.

Long journeys also lock capital. Under stable-flow conditions, an export flow lambda and average unavailable interval W imply approximately lambda W of in-flight value [6]. Prepositioned RLD, reciprocal flows and separately funded liquidity can improve usability. Lending, insurance, external-currency exchange and delivery of goods remain additional economic arrangements. The basic payment operation is not atomic exchange of arbitrary distant assets or goods.

12. Conservation and safety argument

Consider a causally consistent cut: every included destination operation includes its causal source operation. U is spendable value across homes, including fee outputs and locked service/insurance balances. F is exported principal plus reserved destination fees not yet imported or recovered. Z is cumulative owner-authorized retirement. I is cumulative finalized Mint rewards, G = 0 and C = 21,000,000 RLD. Copies, capabilities and receipts contribute no monetary amount. Then:

```
U + F = G + I - Z
```

An unused capability is a resource liability, not an RLD credit. Funded escrow is counted once in U and cannot simultaneously be a freely spendable owner balance. A rejected ticket stays in F until recovery. A stale source may still show Exported after import; its observation is not an additional allocation. These are accounting quantities over the causal execution, not instantaneous global totals available to disconnected users.

Lemma 1: local exclusivity. Two sets of 2f + 1 validators among 3f + 1 intersect in at least f + 1 identities. Under the fault bound and PBFT's durable locking and view-change rules, an honest intersection prevents conflicting final commands at the same position. Valid configuration handoff preserves that exclusion. Quorum arithmetic alone is insufficient.

Lemma 2: valid money and class attribution. Reference execution starts from the common zero-issuance budget, verifies active slot work and partitions each consumed class exactly among outputs, fees, exports and retirement. Set equality permits coalescing without losing an exposure; different sets cannot silently merge. Induction on the causal execution graph preserves amounts and exact risk attribution. If the pinned proof relation implements that execution and recursive soundness holds, a successful unconditional receipt with its bound journal establishes the same validity statement without cold replay of every historical event. This is a conditional composition argument, not a substitute for correctness analysis of the guest program or backend. It establishes neither unseen fork exclusion nor current data availability.

Lemma 3: one receiving obligation. Each coupon binds one source, destination and finite terms. Source S[p] becomes Exported(x) or authorized Closed once. That exclusive decision binds the later exact T and x even though the coupon did not predetermine their inputs. Destination R[p] becomes Imported(x), Rejected(x) or source-certified Closed once. Family membership and exact leaf identity prevent capacity reuse; binding a single source excludes a cross-source race. Restart, compaction and succession preserve both maps. A carrier cannot redirect credit or fill a second obligation from one coupon.

Lemma 4: import excludes recovery. J[x] becomes Imported or Rejected, never both under Lemma 1. Recovery requires valid certified rejection and consumes E[x] once. No other destination can import the same ticket. Therefore the exported amount cannot create both destination spendability and source recovery. A signature on conflicting outcomes is an incident, not permission to accept both.

Proposition: payment conservation. Local transfers and service funding redistribute U. Export reduces U by a and increases F by a; source fees remain in U. Import reduces F by a and creates principal plus destination fees totaling a. Rejection changes neither. Recovery reduces F by a and creates (a-r)+r = a. Mint increases U and I by rho; retirement decreases U and increases Z equally. The lemmas exclude duplicate applications of value-creating transitions, so induction preserves the invariant.

Proposition: bounded issuance. Every root's slot space is a disjoint partition of consumed identities, active intervals and in-flight intervals. Mint moves one slot from active to consumed. ClaimRange partitions an active interval exactly; installation or certified recovery relocates the same detached interval once. No operation introduces a new (s,j). Let B be active unissued budget and Q in-flight unissued budget. Then:

```
I + B + Q = C
U + F + B + Q + Z = C
0 <= I <= C
C = 4096 * 1000000 * 0.005126953125 RLD
```

Lost intervals remain unavailable in B or Q; exhausted or retired value does not replenish them. A malicious quorum signing an invalid split is rejected by correct reference execution or a sound receipt verifier. A quorum certifying conflicting individually valid placements violates the exclusive-ordering premise and can still defeat conservation across unsuspecting observers. The cap is a property of valid exclusive histories, not a magical protection against accepted threshold equivocation.

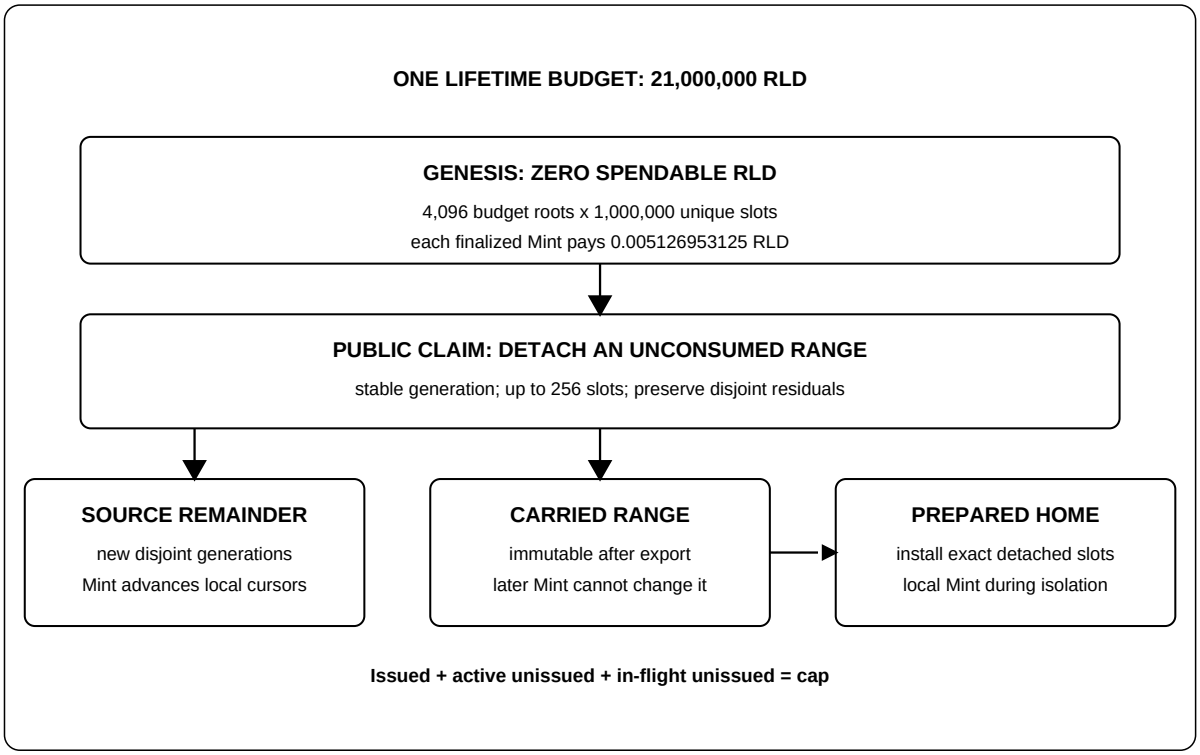


Figure 2. Stable placement generations separate claims from ordinary Mint. A finalized claim detaches one immutable range and retains disjoint residuals; subsequent mining cannot change the exported entitlement.

Proposition: conditional progress and isolation. If a valid source-bound capability exists, a conforming proof-ready export and complete current material eventually arrive, its historical security assumptions remain applicable, a correct funded destination quorum progresses, and rejection has not won first, import completes without a source return message. A rejected payment can recover once after authentic rejection evidence reaches a correct source. Existing local payments need only their own prepared home. Reconnection alone cannot replace a committed local prefix. Permanent authority loss, broken cryptography, unfunded service or missing physical contact invalidates the relevant progress premise; no unconditional deadline or recovery follows.

Proposition: stable placement. Ordinary Mint changes k and mint_anchor, never generation g or placement_anchor. A claim for [a,b) remains locally valid while g is active and k <= a, subject to its other rules. Once finalized, that range is removed from source authority and its immutable ticket contains no later source cursor. Therefore ordinary Mint outside the selected range cannot invalidate its placement work before finalization or its exported entitlement afterwards. A conflicting local split or consumption of a selected slot still defeats an uncommitted claim; it never requires the distant receiver to approve a moving predecessor.

Boundary of the construction. A receiver with the same observed evidence in two worlds cannot act differently merely because one world contains an unseen conflicting certificate. Full execution, short validity proofs, mining puzzles and collateral do not change that observation. Eliminating all accepted remote ordering assumptions would require another conflict-resolution construction with its own delay and fault model. We do not claim to obtain global double-spend exclusion, unrestricted disconnected spending and automatic acceptance of arbitrary authorities simultaneously.

13. Configuration, keys and long journeys

A validator handoff is a special committed command. It binds old/new configurations, exact activation boundary, predecessor, complete monetary state, complete class tables and exposure sets, imports, rejections, capability-use/closure maps, outstanding reservations, service liabilities, issuance generations, cursors and both anchor types. The old configuration commits it; a new quorum authenticates and durably installs the fully execution-verified current state and certified receipts before acting. Old keys cannot authorize the old configuration beyond that boundary. Copying balances alone is not succession. Without an authorized old handoff, an administrator cannot resume stopped money under a replacement root.

For long-journey ordering certificates we describe a finite per-height key schedule using ordinary Ed25519 [7], SHA256 commitments and HKDF-SHA256 [8]. It is a construction requiring secure erasure and custody analysis, not a claim of a newly standardized forward-secure signature. Each validator's epoch descriptor commits to a Merkle root of K = 65536 per-height public keys. Leaves bind C_RLD, domain, validator, epoch, offset and public key; internal nodes use a distinct tag. The epoch fixes the first sequence position; the offset cannot be chosen by the sender.

For initial random secret S_0, define S_(i+1) = SHA256(enc("RLD-KEY-NEXT", binding, i, S_i)). Derive the Ed25519 seed for i with HKDF-SHA256 from S_i, using the epoch binding as salt and a distinct key-purpose information string. The public tree is prepared before epoch activation. The private signer retains only its current S_i, exact locks and response journal, never a recoverable copy of retired S_i or the initial seed. Each PBFT signature carries its exact offset and Merkle authentication path. Signature inputs include phase, view, sequence, predecessor and body, so a path cannot rebind a vote.

After a position is irreversibly committed, the signer durably retains the commit, lock/head and exact already signed responses, advances to S_(i+1), and erases S_i and derived private material before signing a later position. Retransmission uses retained bytes, not regenerated retired signatures. Current-secret compromise should not recover earlier secrets under the hash/KDF assumptions; it can compromise future keys until a valid succession. Consensus safety still needs the active fault bound, secure heads, prevention of concurrent cloned signers and verified erasure. Ordinary disk deletion, encrypted old backups or a local timestamp do not prove those conditions.

The key schedule has finite capacity. A valid old/new handoff must activate a fresh independently generated tree before exhaustion; otherwise signing stops. Epoch identities never reset a source, capability or import namespace. Current key descriptors and authentication paths travel with relevant material; historical handoff validity is folded into the execution receipt, while audit archives retain the history. The erasure assumption narrows later key-theft risk; it does not stop an already compromised quorum from signing conflicting present histories, or signatures being broken by later cryptanalysis. Forward-secure research makes the distinction explicit [9].

Every journey declares supported cryptographic suites and an acceptance horizon. Unknown or cryptographically unsound evidence refuses or remains pending. Post-quantum standards can inform an explicitly instantiated successor [10], but a new algorithm name does not renew all old history. Revocation warnings propagate causally. Secure long-term operation requires generational maintenance and preserved anchors, not a promise of eternal signatures.

Owners choose recovery policies before loss, such as independent threshold keys. Recovery requires an ordinary certified command at the current home; a relay or founder cannot reset ownership. Backups must reconstruct current certified state, permanent decisions and freshness anchors before signing or voting. A restored old image cannot declare itself current. Loss of all state and independent anchors is not repairable by a protocol assertion.

The transparent baseline exposes amounts, keys, current exposure sets and some timing and route metadata. Encrypted carriage does not hide all linkage. Privacy systems such as Zef [11] address a different problem; integration needs a proof that hidden amounts, acceptance predicates and duplicate prevention remain sound.

14. Capacity, archives and failure bounds

Independent domains add local capacity by operating on distinct funds. Throughput is bounded by sustained execution, PBFT, proof generation/verification, current-state access, coupon admission and archival writes. A common hot destination remains a bottleneck. One hundred million completions per day is approximately 1157.41 per second; this arithmetic is not achieved performance. Usable receipt includes exact output validation and durable current material. New stellar information includes physical distance and contact scheduling.

Three costs must be distinguished. Work is consumed once per Mint or claim and must be checked by the execution program. Live monetary components are grouped by exact risk set, not each work event. Historical execution is recursively folded into receipts instead of retransmitted to each cold receiver. For n historical events and c current classes with t deduplicated tags, carried material depends on the supported receipt, current QC, inclusion paths and current c/t data, rather than an obligatory n-entry Mint-origin vector. No constant bound on c, t or proof-generation cost is implied.

The selected succinct backend bounds historical proof representation through recursion, but exact encoded receipt sizes, verification/proving cycles and memory require the pinned implementation and measurements. Each capability fixes maximum receipt bytes, QC/key-path/signature count, class/tag counts, current witness bytes, verification work and durable terminal-state capacity. Source admission also bounds proving work, outstanding candidates and unproved native suffix length. If it cannot construct conforming material before final debit, it waits, retries locally, partitions the payment or refuses. It must not export under an assumed future optimization or require a new stellar approval merely because local inputs changed.

Complete current material remains indispensable. A compact proof with a missing class table, state access witness or accepted key descriptor is not an onward-spendable payment. Domains retain authenticated live state, predecessor receipts and necessary access data so they can create the next execution proof after disconnection. A lost prover's private scratch transcript need not be an owner's dependency, but losing all current state/witnesses can halt progress even while the historical receipt still verifies. Capability funding covers these distinct bytes and operations.

Permanent duplicate-prevention decisions have permanent logical cost. Ordered authenticated maps may support current inclusion/absence witnesses; their small roots do not eliminate the data needed to update them. At 100 million 64-byte decisions daily, raw growth is about 2.34 terabytes per year before indexes and replicas. Coupon Merkle roots similarly compress publication without erasing finite per-leaf obligations. Archival sharding and paid witness services can distribute cost only under explicit availability and authenticated handoff rules, not by deleting undecided or consumed identities.

Full historical audit records remain useful for forensic review, implementation equivalence, proof regeneration and cryptographic renewal. A verified recursive receipt can replace old replay material for bounded current payment validation under its original assumptions. It cannot make deleted bytes recoverable, reprove a history under a broken old proof system or hide an incident behind an obsolete checkpoint. Services state exactly which current data, receipts and historical archives they retain, their retrieval bounds, independent copies and funded renewal obligations. A future proof-suite handoff must renew valid current state while the predecessor suite is still trustworthy; it cannot retroactively cure forgery.

Compaction preserves monetary roots, all permanent decisions, current exposures, allocation partitions and unresolved liabilities. Succession transfers current state and receipts plus contracted audit custody, funding and independent retrieval information before old service ends. Fake receipts, unsupported programs, conditional assumptions, lost bytes, rollback

and out-of-bound packages refuse or remain explicitly pending. No silent full-replay fallback, raised resource limit or quorum-signed snapshot can masquerade as the promised compact profile.

The design exchanges repeated historical carriage for a substantial proving obligation and new soundness assumptions. It exchanges event-level output attribution for exact exposure-level grouping, not universal fungibility. Funded admission, replicated current state, independent audit archives and prepared succession reduce practical risks; finite reserves, software correctness and physical contacts still need their own evidence.

15. Relation to prior work and scope of the contribution

Bitcoin demonstrates open-work selection of a monetary history [1]. PBFT supplies local state-machine replication under explicit validator assumptions [2]. This design keeps conflict resolution local to avoid a distant branch contest over an isolated community's completed payments; it changes the trust model rather than improving every property at once.

FastPay and Zef show that payment workloads can avoid a single global total order [11,12]. Asynchronous asset-transfer research separates transfer from general consensus [13]. Escrow and bounded-counter techniques partition finite rights before disconnected use [14]. Those database results do not supply Byzantine monetary security. Here work entitlements additionally require authenticated exclusive histories, valid execution and permanent import/placement decisions.

IBC separates remote consensus verification, transport and application transitions [15]. Cross-shard work studies replay and stale-message failures [16]. Delay-tolerant networking supplies the appropriate carriage model [4,5]. Recursive zkVM composition provides a concrete route to execution receipts [17,18]; it does not supply RLD monetary semantics automatically. We do not claim a new BFT algorithm, proof primitive, general transfer concept or invention of store-carry-forward networking.

The contribution proposed here is their explicit composition for one RLD currency across extremely delayed contacts: independently verified monetary execution, advance source-bound capabilities, proof-ready destination-bound exports, rejection-only recovery, exact exposure-class amounts, stable-generation divisible work entitlements and finite proof/key/archive succession. Its novelty and security require independent comparison and analysis. Interstellar terminology alone establishes neither.

The most immediate applications are already-separated communities, moving platforms and disrupted infrastructure with real custody and payment needs. Whether users prefer RLD to existing money depends on practical costs, acceptance, access and risk. A payment mechanism does not itself create demand, stable purchasing power or social impact. Physical links, sustainable institutions and useful adoption remain necessary.

16. Conclusion

Interstellar payment can preserve local economic activity by assigning exclusive spend authority before disconnection. A destination preissues source-bound capacity without choosing the payer's future inputs. The source selects and proves the exact payment locally, surrenders its rights, and carries a certified capsule forward. One import or permanent rejection follows. Onward local spending needs no source reply, and silence never creates a refund.

runlai (RLD) remains the sole native currency with a twenty-one-million lifetime cap and zero initial allocation. Ordinary Mint consumes fixed slots while preserving placement generations; a finalized claim detaches an immutable range whose validity is independent of later source mining. Exact exposure classes coalesce compatible amounts without erasing real trust. Recursive execution receipts remove an obligatory full-history replay from cold payment receipt while keeping complete current data necessary for future spending.

These changes address concrete preparation and scale failures without claiming unconditional security or world impact. Accepted exclusive ordering, correct and sound proof programs, current-data availability, finite cryptography and sustainable funding remain premises. Arbitrary authorities are not automatically trusted and destroyed domains can strand value. Under the stated conditions, prepared communities can continue local settlement and exchange owner-authorized RLD across delayed one-way contacts without a continuously reachable center.

References

[1] S. Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. https://bitcoin.org/bitcoin.pdf

[2] M. Castro and B. Liskov. Practical Byzantine Fault Tolerance, OSDI, 1999. https://pdos.csail.mit.edu/6.824/papers/castro-practicalbft.pdf

[3] A. Biryukov, D. Dinu, D. Khovratovich and S. Josefsson. Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications, RFC 9106, 2021. https://www.rfc-editor.org/rfc/rfc9106.html

[4] S. Burleigh, K. Fall and E. Birrane. Bundle Protocol Version 7, RFC 9171, 2022. https://www.rfc-editor.org/rfc/rfc9171.html

[5] E. Birrane and K. McKeever. Bundle Protocol Security (BPSec), RFC 9172, 2022. https://www.rfc-editor.org/rfc/rfc9172.html

[6] J. D. C. Little. A Proof for the Queuing Formula: L = lambda W, Operations Research, 1961. https://doi.org/10.1287/opre.9.3.383

[7] S. Josefsson and I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA), RFC 8032, 2017. https://www.rfc-editor.org/rfc/rfc8032.html

[8] H. Krawczyk and P. Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF), RFC 5869, 2010. https://www.rfc-editor.org/rfc/rfc5869.html

[9] M. Bellare and S. K. Miner. A Forward-Secure Digital Signature Scheme, CRYPTO, 1999. https://cseweb.ucsd.edu/~mihir/papers/fsig.pdf

[10] NIST. Module-Lattice-Based Digital Signature Standard, FIPS 204, 2024. https://doi.org/10.6028/NIST.FIPS.204

[11] M. Baudet, A. Sonnino, M. Kelkar and G. Danezis. Zef: Low-latency, Scalable, Private Payments, 2022. https://arxiv.org/abs/2201.05671

[12] M. Baudet, G. Danezis and A. Sonnino. FastPay: High-Performance Byzantine Fault Tolerant Settlement, 2020. https://arxiv.org/abs/2003.11506

[13] P. Kuznetsov, Y.-A. Pignolet, P. Ponomarev and A. Tonkikh. Permissionless and Asynchronous Asset Transfer, DISC, 2021. https://doi.org/10.4230/LIPIcs.DISC.2021.28

[14] V. Balegas, D. Serra, S. Duarte, C. Ferreira, R. Rodrigues, N. Preguica, M. Shapiro and M. Najafzadeh. Extending Eventually Consistent Cloud Databases for Enforcing Numeric Invariants, 2015. https://arxiv.org/abs/1503.09052

[15] Inter-Blockchain Communication. How the IBC Protocol Works. https://ibcprotocol.dev/how-ibc-works

[16] A. Sonnino, S. Bano, M. Al-Bassam and G. Danezis. Replay Attacks and Defenses Against Cross-shard Consensus in Sharded Distributed Ledgers, 2020. https://arxiv.org/abs/1901.11218

[17] RISC Zero. SuccinctReceipt, risc0-zkvm 3.0.6 API documentation. https://docs.rs/risc0-zkvm/3.0.6/risc0_zkvm/struct.SuccinctReceipt.html

[18] RISC Zero. Proof Composition, zkVM 3.0 documentation. https://dev.risczero.com/api/zkvm/composition